

TypeScript - Principiante

Tema 11: Herencia

20 ejercicios progresivos, sin soluciones, para practicar herencia entre clases, extends, super, sobrescritura de metodos, protected y especializacion de comportamiento.

Ejercicio 1 - Persona y empleado

Crea una clase Person con name y age. Crea Employee heredando de Person y agrega employeeId.

Requisitos

- Usa extends.
- Inicializa la parte heredada correctamente.
- Crea una instancia de Employee.

Ejemplo de ejecucion

100 - Ana - 28 anos

Objetivo

Practicar herencia basica.

Pista

La clase hija hereda miembros accesibles de la clase padre.

Ejercicio 2 - Constructor heredado

Crea Vehicle con brand y model, y Car con numberOfDoors.

Requisitos

- Car debe usar super en su constructor.
- Muestra todos los datos.

Ejemplo de ejecucion

Toyota Corolla - 4 puertas

Objetivo

Practicar super en constructores.

Pista

super permite inicializar la parte correspondiente a la clase base.

Ejercicio 3 - Metodo heredado

Crea Animal con un metodo eat. Crea Dog heredando de Animal y agrega bark.

Requisitos

- Dog debe poder usar eat sin redefinirlo.
- Ejecuta ambos metodos.

Ejemplo de ejecucion

El animal esta comiendo

Guau

Objetivo

Reutilizar comportamiento heredado.

Pista

Los metodos publicos se heredan.

Ejercicio 4 - Sobrescribir metodo

Crea Animal con makeSound y crea Dog y Cat sobrescribiendo ese metodo.

Requisitos

- Usa override si tu configuracion lo requiere.
- Cada clase devuelve un sonido diferente.

Ejemplo de ejecucion

Dog: Guau

Cat: Miau

Objetivo

Practicar method overriding.

Pista

Una subclase puede proporcionar su propia implementacion.

Ejercicio 5 - Informacion de empleado

Crea Person con getInfo y Employee que sobrescriba getInfo agregando departamento.

Requisitos

- Reutiliza cuando sea conveniente la informacion de la clase padre.
- Devuelve string.

Ejemplo de ejecucion

Ana - 28 - Desarrollo

Objetivo

Practicar especializacion de metodos.

Pista

Puedes invocar super.getInfo() desde la clase hija.

Ejercicio 6 - Cuenta de ahorro

Crea BankAccount con balance y SavingsAccount con interestRate.

Requisitos

- balance debe ser protected.
- SavingsAccount agrega applyInterest.
- No accedas directamente al saldo desde fuera.

Ejemplo de ejecucion

Saldo despues de intereses: \$10500

Objetivo

Combinar herencia y protected.

Pista

protected permite que la subclase utilice el estado.

Ejercicio 7 - Producto digital

Crea Product con id, name y price. Crea DigitalProduct con fileSize.

Requisitos

- Hereda las propiedades comunes.
- Agrega informacion especifica.

Ejemplo de ejecucion

Curso TypeScript - \$500 - 2 GB

Objetivo

Evitar duplicacion mediante herencia.

Pista

Coloca en la clase base solo lo verdaderamente comun.

Ejercicio 8 - Producto fisico

Partiendo de Product, crea PhysicalProduct con weight y stock.

Requisitos

- Agrega un metodo para verificar disponibilidad.
- No modifiques Product innecesariamente.

Ejemplo de ejecucion

Monitor - Disponible: true

Objetivo

Practicar varias especializaciones de una misma clase base.

Pista

DigitalProduct y PhysicalProduct pueden heredar de Product.

Ejercicio 9 - Figuras

Crea Shape con un metodo getArea. Crea Rectangle y Circle que proporcionen su propio calculo.

Requisitos

- Cada clase debe calcular su area.
- Tipa los retornos.

Ejemplo de ejecucion

Rectangulo: 50

Circulo: 78.54

Objetivo

Practicar comportamiento especializado.

Pista

La misma operacion puede tener implementaciones distintas.

Ejercicio 10 - Usuarios por rol

Crea User con id y name. Crea AdminUser y GuestUser.

Requisitos

- AdminUser agrega permisos administrativos.
- GuestUser tiene acceso limitado.
- Agrega un metodo getPermissions en cada tipo.

Ejemplo de ejecucion

Admin: create, edit, delete

Guest: view

Objetivo

Modelar especializaciones mediante herencia.

Pista

El comportamiento comun puede vivir en User.

Ejercicio 11 - Notificaciones

Crea Notification con recipient y send. Crea EmailNotification y SmsNotification sobrescribiendo send.

Requisitos

- Cada subclase necesita informacion especifica.
- Muestra mensajes diferentes.

Ejemplo de ejecucion

Email enviado a user@test.com

SMS enviado a 5551234

Objetivo

Practicar polimorfismo basado en herencia.

Pista

Todas las notificaciones comparten una operacion send.

Ejercicio 12 - Empleados

Crea Employee con salary y calculatePay. Crea HourlyEmployee y SalariedEmployee.

Requisitos

- Cada tipo calcula su pago de manera diferente.
- Evita duplicar datos comunes.

Ejemplo de ejecucion

Pago por hora: \$4800

Pago fijo: \$18000

Objetivo

Modelar reglas distintas con una clase base comun.

Pista

La subclase decide como calcular el pago.

Ejercicio 13 - Descuento

Crea Discount con apply y subclases PercentageDiscount y FixedDiscount.

Requisitos

- Ambas reciben un precio.
- No permitas resultados negativos.
- Cada una aplica una regla diferente.

Ejemplo de ejecucion

Precio final: \$850

Objetivo

Practicar estrategias sencillas mediante herencia.

Pista

Sobrescribe el comportamiento que varia.

Ejercicio 14 - Repositorio base

Crea una clase base Repository que mantenga una coleccion protegida de elementos simples. Crea UserRepository heredando de ella.

Requisitos

- Repository permite listar elementos.
- UserRepository agrega busqueda por nombre.
- Usa protected donde corresponda.

Ejemplo de ejecucion

Usuarios encontrados: 2

Objetivo

Practicar reutilizacion en clases de infraestructura.

Pista

La subclase puede agregar operaciones especificas.

Ejercicio 15 - Excepciones de dominio simples

Crea una clase AppError que herede de Error y agrega una propiedad code. Crea ValidationError heredando de AppError.

Requisitos

- Usa super correctamente.
- Genera y captura un ValidationError.

Ejemplo de ejecucion

VALIDATION_ERROR: Nombre requerido

Objetivo

Practicar herencia con clases existentes.

Pista

Error tambien puede extenderse.

Ejercicio 16 - Transporte

Crea Transport con move. Crea Car, Bicycle y Airplane.

Requisitos

- Cada clase sobrescribe move.
- Guarda distintas instancias en una coleccion comun.
- Ejecuta move para todas.

Ejemplo de ejecucion

Auto circulando

Bicicleta avanzando

Avion volando

Objetivo

Introducir uso polimorfico de clases heredadas.

Pista

Una referencia del tipo base puede contener objetos derivados.

Ejercicio 17 - Sistema de archivos conceptual

Crea FileSystemItem con name y size. Crea File y Folder.

Requisitos

- File agrega extension.
- Folder contiene FileSystemItem[].
- Calcula el tamaño total de una carpeta.

Ejemplo de ejecución

Carpeta docs - 2500 KB

Objetivo

Practicar estructuras jerárquicas.

Pista

Una carpeta puede contener elementos del tipo base.

Ejercicio 18 - Pedidos especializados

Crea Order con id, items y calculateTotal. Crea OnlineOrder y StoreOrder.

Requisitos

- OnlineOrder agrega shippingCost.
- StoreOrder no cobra envío.
- Sobrescribe el cálculo cuando sea necesario.

Ejemplo de ejecución

Pedido online: \$2300

Pedido tienda: \$2000

Objetivo

Aplicar herencia a reglas de negocio.

Pista

Reutiliza el cálculo base con super cuando tenga sentido.

Ejercicio 19 - Personal de empresa

Crea Employee y especializaciones Developer, Manager y SalesPerson.

Requisitos

- Todos comparten id, name y baseSalary.
- Cada tipo calcula un bono diferente.
- Crea una lista Employee[] con distintos tipos.
- Calcula el costo total de nomina.

Ejemplo de ejecucion

Nomina total: \$85000

Objetivo

Practicar polimorfismo y sobrescritura.

Pista

Invoca el mismo metodo sobre objetos de diferentes subclases.

Ejercicio 20 - Sistema de pagos

Construye Payment como clase base y CreditCardPayment, BankTransferPayment y CashPayment como clases derivadas.

Requisitos

- Payment contiene amount y una operacion process.
- Cada subclase implementa reglas propias.
- Tarjeta contiene ultimos digitos.
- Transferencia contiene referencia bancaria.
- Efectivo calcula cambio cuando corresponda.
- Guarda distintos pagos en una coleccion Payment[].
- Procesa todos usando la misma operacion.
- Usa protected cuando una subclase necesite datos de la base.
- No uses any.

Ejemplo de ejecucion

Tarjeta procesada: \$1200

Transferencia procesada: \$800

Efectivo procesado: \$500

Objetivo

Integrar herencia, sobrescritura, protected y polimorfismo.

Pista

Define primero que informacion y comportamiento son comunes a todos los pagos.